UNIT-3

1. Explain the concepts of local receptive field and parameter sharing in CNNs. Show how do these concepts make CNNs suitable for image data?

2. Explain the influence of kernel size, stride, padding, and number of filters on feature extraction capability and computational complexity of a CNN.

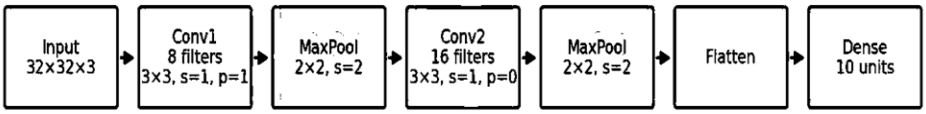
3.Explain the purpose of pooling layers in CNNs. Compare Max Pooling and Average Pooling.

4.Explain padding and stride in CNNs. Discuss how they influence output dimensions and the amount of spatial information retained in feature maps.

5.Explain how CNNs learn hierarchical visual features. Describe the type of features generally extracted in shallow, intermediate, and deeper convolutional layers.

6.Explain transfer learning. Discuss the difference between feature extraction and fine-tuning when adapting a pre-trained CNN to a new task.

7.The convolutional network shown below processes an RGB image. Use O = [(N − F + 2P)/S]+ 1 for every convolution and pooling stage.



Evaluate:

(i) The output volume produced by each of the five stages up to the Flatten operation.

(ii) The number of trainable parameters in Conv1 and in Conv2.

(iii) The size of the flattened feature vector.

(iv) The number of parameters in the Dense layer, and the total for the whole network. Comment on which layer dominates the parameter count and why.

8. An RGB image of size 128 × 128 × 3 is passed through:

Layer 1: Convolution with 16 filters, kernel 3 × 3, stride 1, padding 1; Layer 2: Max Pooling with pool size 2 × 2 and stride 2;

Layer 3: Convolution with 32 filters, kernel 5 × 5, stride 1, padding 2; Layer 4: Max Pooling with pool size 2 × 2 and stride 2;

Layer 5: Flatten.

(i) Develop the block architecture. (ii) Evaluate output volume after every layer. (iii) Evaluate trainable parameters in both convolution layers. (iv)Evaluate flattened feature vector size. Use O = floor ((N - F + 2P)/S) + 1.

9. For an image consider:

Input 64 × 64 × 3 → Conv1: 3 × 3, 8 filters, stride 1, padding 1 → Max Pooling: 2 × 2, stride 2 → Conv2: 3 × 3, 16 filters, stride 1, padding 1 → Max Pooling: 2 × 2, stride 2 → Conv3: 3 × 3, 32 filters, stride 1, padding 1 → Flatten.

Develop architecture and Evaluate (i) output volume after every layer, (ii) trainable parameters in Conv1, Conv2, and Conv3, and (iii) final flattened vector size.

10.An input image of size 100 × 100 × 3 passes through:

Conv1: 10 filters, 5 × 5 kernel, stride 1, padding 2 → Max Pooling: 2 × 2, stride 2

Conv2: 20 filters, 3 × 3 kernel, stride 1, padding 1 → Max Pooling: 2 × 2, stride 2

Conv3: 40 filters, 3 × 3 kernel, stride 1, padding 1 → Flatten.

(i) Illustrate the architecture.
(ii) Evaluate output volume after each layer.
(iii) Estimate parameters in Conv1, Conv2, and Conv3.

Evaluate flattened vector size.

11.An image classification CNN is:

224 × 224 × 3 → Conv1: 16 filters, 3 × 3, stride 1, padding 1 → Max Pooling: 2 × 2, stride 2 → Conv2: 32 filters, 3 × 3, stride 1, padding 1 → Max Pooling: 2 × 2, stride 2 → Conv3: 64 filters, 3 × 3, stride 1, padding 1 → Flatten.

(i) Draw block architecture.

(ii) Evaluate output volume after every layer.

(iii) Compute total convolutional parameters.

(iv) Evaluate final flattened vector size.

12. A CNN receives input 64 × 64 × 3. It contains
Conv1: 8 filters, 7 × 7 kernel, stride 2, padding 3;
Conv2: 16 filters, 3 × 3 kernel, stride 1, padding 1.
Evaluate:

(i) Output volume after Conv1,
(ii) Output volume after Conv2,
(iii) Trainable parameters in Conv1, and Trainable parameters in Conv2.